

Sequenzanalyse 2

by Jens Stoye

Summer semester 2026

Lecture notes by Liliana Sanfilippo

April 2026

Contents

1	Sequence Comparison Beyond Edit Distance	5
1.1	The q -gram Distance	5
1.1.1	Practical Computation	6
1.1.2	Choice of q	8
1.1.3	Efficiency	9
1.1.4	Relation to the Edit Distance	10
2	Practical Aspects of de Bruijn Graphs	11
2.1	Definition and Basic Properties	11
2.2	Words with the same $(k + 1)$ -mer Profile	11
2.3	Variants of de Bruijn Graphs	13
2.4	Implementation of de Bruijn Graphs: Sets of k -mers	16
	Backmatter	i
	Definitionen	i
	Figures	iii
	Index	v
	References	v

CHAPTER 1

Sequence Comparison Beyond Edit Distance

The **edit distance** (or **alignment distance**) is one of the most fundamental ways of quantifying the dissimilarity of two sequences x and y over a finite alphabet Σ . It is based on the cost of an alignment $A = (A_1, A_2, \dots, A_n)$, defined as the sum of the costs of A 's columns, i.e.,

$$\text{cost}(A) = \sum_{i=1}^n \text{cost}(A_i),$$

where the cost of alignment column $A_i = \begin{pmatrix} a_i \\ b_i \end{pmatrix}$, $a_i, b_i \in \Sigma$, in the simplest (unit cost) case is 0 for a match column ($a_i = b_i$), and 1 for a mismatch column ($a_i \neq b_i$) or an indel column ($a_i = -$ or $b_i = -$). Then we have:

Definition 1.1 alignment distance

The alignment distance of two sequences $x, y \in \Sigma^*$ is defined as $d(x, y) := \min\{\text{cost}(A) : A \in \mathcal{A}^*$ is an alignment of x and $y\}$

The corresponding computational problem is as follows:

Problem 1.1 Alignment Problem

For two given strings $x, y \in \Sigma^*$ and a given cost function, find the alignment distance of x and y and one or all optimal alignment(s).

From the “Sequence Analysis 1” lecture we know that this problem can be solved in $O(mn)$ time using a simple and elegant dynamic programming algorithm, where $m = |x|$ and $n = |y|$ are the two sequence lengths.

However, there are situations in which edit distance and alignments are not the perfect model to compare sequences. In the following two sections we will study two interesting alternatives.

1.1 | The q -gram Distance

Edit distances emphasize the correct order of the characters in a string. If, however, a large block of a text is moved somewhere else (e.g. two paragraphs of a text are exchanged), the edit distance will be high, although from a certain standpoint, the texts are still quite similar. A more appropriate notion of distance can be defined in several ways. Here we introduce the **q -gram distance** d_q , which is actually a pseudo-metric: There exist strings $x \neq y$ with $d_q(x, y) = 0$.

Remember that for $q \in \mathbb{N}$, a **q -gram** is a string of length q . *Wir reden hier über überlappende q -gramme!*

Definition 1.2 occurrence count

Let $x \in \Sigma^m$ and choose $q \in [1, m]$. The occurrence count of a q -gram $z \in \Sigma^q$ in x is the number

$$\text{Occ}(x, z) := |\{i \in [1, m - q + 1] : x[i \dots i + q - 1] = z\}|$$

Definition 1.3 **q-gram profile**

Let $\sigma = |\Sigma|$. The q-gram profile of x is a σ^q -element vector $p_q(x) := (p_q(x)_z)_{z \in \Sigma^q}$, indexed by all q-grams, defined by $p_q(x)_z := \text{Occ}(x, z)$

Definition 1.4 **q-gram distance**

The q-gram distance of $x \in \Sigma^m$ and $y \in \Sigma^n$ is defined for $1 \leq q \leq \min\{m, n\}$ as

$$d_q(x, y) := \sum_{z \in \Sigma^q} |p_q(x)_z - p_q(y)_z|$$

Example 1.1 Consider $\Sigma = A, B$, $x = ABAA$, $y = ABAB$, $z = AABA$, and $q = 2$. Using lexicographic order (AA, AB, BA, BB) among the **q-grams**, we have $p_2(x) = (1, 1, 1, 0)$, $p_2(y) = (0, 2, 1, 0)$ and $p_2(z) = (1, 1, 1, 0)$. Therefore we obtain $d_2(x, y) = 2$ and $d_2(x, z) = 0$ (although $x \neq z$).

Theorem 1.1 The **q-gram distance** d_q is a pseudo-metric.

Proof. It fulfills nonnegativity, symmetry and the triangle inequality (exercise).

1.1.1 | Practical Computation

If the **q-gram profile** of a string $x \in \Sigma^m$ is (i.e., if it does not contain mostly zeros), it makes sense to implement it as an array $p[0 \cdots \Sigma^q - 1]$, where $p[i]$ counts the number of occurrences of the i -th **q-gram** in some arbitrary but fixed (e.g. lexicographic) order.

Definition 1.5 **ranking function**

A bijective function $r : \mathcal{X} \rightarrow [0, |\mathcal{X}| - 1]$ for a finite set $\mathcal{X}$

Kommentar 1.1 Warum $|\mathcal{X}| - 1$?

Definition 1.6 **ranking algorithm**

An algorithm that implements a ranking function

Definition 1.7 **unranking function**

The inverse of a ranking function

Definition 1.8 **unranking algorithm**

An algorithm that implements a unranking function

Of course, we are interested in an efficient (un-)ranking algorithm for the set of all **q-grams** over Σ . A classical one is to first define a **ranking function** r_Σ on the characters (alphabet encoding) and then extend it to a **ranking function** r on the **q-grams** by interpreting them as base- σ numbers. There are two possibilities to number the positions of a **q-gram**: From q to 1 falling or from 1 to q rising, as shown in the Example 1.2.

Kommentar 1.2 Was sind base- σ numbers?

Example 1.2 Assume the alphabet $\Sigma = A, C, G, T$ and the alphabet encoding $r_\Sigma(A) = 0$, $r_\Sigma(C) = 1$, $r_\Sigma(G) = 2$, and $r_\Sigma(T) = 3$. For $q = 5$, the two different possibilities to rank the **q-gram** $z = \text{ATACG}$ are:

(a) Falling ranking

$$\begin{aligned} r(ATACG) &= \underbrace{r_{\Sigma}(z[1]) \cdot \sigma^4}_{0 \cdot 4^4} + \underbrace{r_{\Sigma}(z[2]) \cdot \sigma^3}_{3 \cdot 4^3} + \underbrace{r_{\Sigma}(z[3]) \cdot \sigma^2}_{0 \cdot 4^2} + \underbrace{r_{\Sigma}(z[4]) \cdot \sigma^1}_{1 \cdot 4^1} + \underbrace{r_{\Sigma}(z[5]) \cdot \sigma^0}_{2 \cdot 4^0} \\ &= 0 + 192 + 0 + 4 + 2 = 198 \end{aligned}$$

(b) Rising ranking

$$\begin{aligned} r(ATACG) &= \underbrace{r_{\Sigma}(z[1]) \cdot \sigma^0}_{0 \cdot 4^0} + \underbrace{r_{\Sigma}(z[2]) \cdot \sigma^1}_{3 \cdot 4^1} + \underbrace{r_{\Sigma}(z[3]) \cdot \sigma^2}_{0 \cdot 4^2} + \underbrace{r_{\Sigma}(z[4]) \cdot \sigma^3}_{1 \cdot 4^3} + \underbrace{r_{\Sigma}(z[5]) \cdot \sigma^4}_{2 \cdot 4^4} \\ &= 0 + 12 + 0 + 64 + 512 = 588 \end{aligned}$$

The first numbering of positions in (a) corresponds naturally to the ordering of positional number systems; in the decimal number 23, the first 2 has positional value (weight) $10^1 = 10$ whereas the 3 has the lower weight of $10^0 = 1$. For texts, however, the second numbering in (b) is more natural since we expect lower position numbers on the left side of higher position numbers.

In general, we see that $z \in \Sigma^q$ has rank

$$(a) : r(z) = \sum_{i=1}^q r_{\Sigma}(z[i]) \cdot \sigma^{q-i}, (b) : r(z) = \sum_{i=1}^q r_{\Sigma}(z[i]) \cdot \sum_{j=0}^{i-1} \sigma^j.$$

Whichever numbering system we use, we have to do it consistently.

For each of the two ways to rank a q -gram, there is a corresponding unranking method. Both methods work with integer division and remainder. The one for the falling ranking function uses a dynamic divisor and determines the characters of the q -gram based on the integer results, the other one for the rising ranking function uses a fixed divisor and determines the characters based on the remainder.

Example 1.3 Unranking the rank values 198 and 588 from Example 1.2.

(a) Unranking for falling ranking function

$$\begin{aligned} \frac{198}{4^4} &= 0, \quad R \ 198 \rightarrow A \\ \frac{198}{4^3} &= 3, \quad R \ 6 \rightarrow T \\ \frac{6}{4^2} &= 0, \quad R \ 6 \rightarrow A \\ \frac{6}{4^1} &= 1, \quad R \ 2 \rightarrow C \\ \frac{2}{4^0} &= 0, \quad R \ 0 \rightarrow G \end{aligned}$$

(b) Unranking for rising ranking function

$$\begin{aligned}\frac{588}{4} &= 147, & \text{R } 0 &\rightarrow A \\ \frac{147}{4} &= 36, & \text{R } 3 &\rightarrow T \\ \frac{36}{4} &= 9, & \text{R } 0 &\rightarrow A \\ \frac{9}{4} &= 2, & \text{R } 1 &\rightarrow C \\ \frac{2}{4} &= 0, & \text{R } 2 &\rightarrow G\end{aligned}$$

Kommentar 1.3 To example 1.3.

$$\frac{198}{4^4} = 0, \quad \text{R } 198 \rightarrow A$$

Because $\frac{198}{4^4} = \frac{198}{256} \approx 0,77$ which means the rest (R) is 198, since $198 - 0 = 198$. We decode it into A, since we are using the result of the fraction to find the letters: $r_{\Sigma}(A) = 0$.

$$\frac{588}{4} = 147, \quad \text{R } 0 \rightarrow A$$

Because with the rising function we are using the rest to find the letters instead of the result of the fraction.

Both methods need q iterations and reconstruct the q -gram from the first position to the last. Each iteration takes a starting value r and determines the corresponding character of the q -gram while updating r for the next iteration. The starting value for the first iteration is the rank of the q -gram. In iteration i in the unranking method (a), for $1 \leq i \leq q$, the starting value r is divided by σ^{q-i} . The integer result c of the division determines the decoded character as described by r_{Σ} of the used ranking function. The remainder r' of the division serves as starting value for the next iteration.

$$(a) \quad \frac{r}{\sigma^{q-1}} = c, \quad \text{R } r'$$

In unranking method (b), the starting value r is always divided by σ . Here, the remainder c of the integer division decodes the corresponding character and the integer result r is the starting value for the next iteration.

$$(b) \quad \frac{r}{\sigma} = r', \quad \text{R } c$$

1.1.2 | Choice of q

In principle, we could use any $q \in [1, \min\{m, n\}]$ (Any q that is only as long as the shorter of the two strings being compared) to compare two strings of lengths m and n . In practice, some choices are more reasonable than others.

For example, for $q = 1$, we merely add the differences of the single letter counts. If m and n are large, there are many (even very differently looking) strings with the same letter counts, which would all have distance zero from each other.

If on the other hand $q = m \leq n$, and $m \approx n$, the distance d_q (q -gram distance) between the strings is always close to $n - m$, which does not give us a good resolution either. Also, the q -gram profile of both strings is extremely sparse in these cases. We thus ask for a value of q such that each q -gram occurs about once (or $c \geq 1$ times) on average.

Assuming a uniform random distribution of letters in a given string $x \in \Sigma^m$, the probability that a fixed q -gram z starts at a fixed position $i \in [1, m - q + 1]$ is $1/\sigma^q$, where $\sigma = |\Sigma|$.

Kommentar 1.4 It is NOT $1/\sigma$ (1 divided though the number of letters in the alphabet). One might think that, since at that “fixed position” (aka. any position) every letter of the alphabet can occur and if the distribution is uniform every letter has the same chance to occur there so it could start there, right? No. We have to account for the length of the q -gram. The chance that the starting letter of the q -gram is at that position is $1/\sigma$. The chance that the

second letter of the q -gram is in the position thereafter is $1/\sigma$. The chance that the third letter of the q -gram ...and so on. Meaning we have

$$\underbrace{1/\sigma \cdot 1/\sigma \cdots 1/\sigma}_{q \text{ times}} = 1/\sigma^q.$$

The expected number of occurrences of z in x is thus $(m - q + 1)/\sigma^q$.

Kommentar 1.5 Because there are $(m - q + 1)$ possible starting position for a q -gram in a string of length m .

The condition that this number equals c (The number each q -gram occurs approximately) leads to the condition $(m + 1)/c = q/c + \sigma^q$ (see Comment 1.6), or approximately $m/c = \sigma^q$, since $1/c$ and q/c are comparatively small.

Kommentar 1.6 It leads to the condition $(m + 1)/c = q/c + \sigma^q$, because:

$$\begin{aligned} \frac{(m - q + 1)}{\sigma^q} &= c && | \cdot \sigma^q \\ m - q + 1 &= c \cdot \sigma^q && | + q \\ m + 1 &= c \cdot \sigma^q + q && | : c \\ \frac{m + 1}{c} &= \sigma^q + \frac{q}{c} \end{aligned}$$

Thus setting

$$q = \left\lfloor \frac{\log(m)\log(c)}{\log(\sigma)} \right\rfloor$$

ensures an expected occurrence count $\geq c$ of each q -gram.

1.1.3 | Efficiency

Obviously, for $z \in \Sigma^q$, the ranking function $r(z)$ can be computed in $O(q)$ time. Both the falling and the rising ranking functions have the advantage that when a window of length q is shifted along a text, the rank of the q -gram can be updated in $O(1)$ time:

Assume that $t = aub$ with $a \in \Sigma$, $u \in \Sigma^{q-1}$ and $b \in \Sigma$. The first q -gram is au , whereas ub is the updated one. Now we can define an update system for each of the ranking systems above. Let $j = r(au)$. For the falling ranking function we compute:

$$(a) \ r(ub) = (j \bmod \sigma^{q-1}) \cdot \sigma + r_\Sigma(b),$$

Or without using j :

$$(a) \ r(ub) = (r(au) \bmod \sigma^{q-1}) \cdot \sigma + r_\Sigma(b),$$

and for the rising ranking function:

$$(b) \ r(zb) = \left\lfloor \frac{j}{\sigma} \right\rfloor + f(b), \quad \text{where } f(b) = r_\Sigma(b) \cdot \sigma^{q-1}.$$

Or without using j and f :

$$(b) \ r(zb) = \left\lfloor \frac{r(au)}{\sigma} \right\rfloor + r_\Sigma(b) \cdot \sigma^{q-1}.$$

If we take a more detailed look at both systems now, we can see that the second way is possibly more efficient (as it avoids the modulo operation and only uses integer division) if f is implemented as a lookup table $f : \Sigma \rightarrow \mathbb{N}_0$. If $\sigma = 2^k$ for some $k \in \mathbb{N}$, multiplications and divisions can be implemented by bit shifting operators, e.g. in the second system (b), $r(ub) = (j \gg k) + f(b)$.

It is allegedly easy to see that the q -gram corresponding to a ranking value can be obtained in $O(q)$ time for both unranking methods.

Now, to compute $d_q(x, y)$ for $x \in \Sigma^m$ and $y \in \Sigma^n$,

1. initialize integer array $p_q[0 \dots \sigma^q - 1]$ with zeroes

2. for each $i \in [1, mq + 1]$, increment $p_q[r(x[i \cdot i + q1])]$
3. for each $i \in [1, nq + 1]$, decrement $p_q[r(y[i \cdot i + q1])]$
4. initialize $d := 0$,
5. for each $j \in [0, \sigma^q - 1]$, increment d by $|p_q[j]|$

Kommentar 1.7 Explanation of the steps and complexites.

Step	Complexity	Because...
1. Initialise empty array	$O(\sigma^q)$	the array has σ^q entries
2. Count q-grams in x	$O(m)$	compute hash of first q-gram in $O(q)$, then use rolling hash to update
3. Count q-grams in y	$O(n)$	see above
4. Initialize distance	$O(1)$	trivial
5. Summarize the absolute difference	$O(\sigma^q)$	Iteration through σ^q items

Leading to:

$$O(\sigma^q) + O(m) + O(n) + O(\sigma^q) = 2 \cdot O(\sigma^q) + O(m) + O(n) = O(2 \cdot \sigma^q) + O(m) + O(n) = O(\sigma^q) + O(m) + O(n) = O(\sigma^q + m + n)$$

This procedure **very** obviously takes $O(\sigma^q + m + n)$ time. If $m \leq n$ and q is as suggested above, this is $O(n)$ time overall. The space complexity is $O(\sigma^q)$ to store the array p_q .

If q is comparatively large, it does not make sense to maintain a full array. Instead, we maintain a data structure that contains only the nonzero entries of p_q . If $q = O(\log(m + n))$, we can still achieve $O(m + n)$ time using hashing.

1.1.4 | Relation to the Edit Distance

Notiz

Practical Aspects of de Bruijn Graphs

A note on terminology: In Section 1.1 we were speaking of *q*-grams when referring to short (sub)strings of a certain length q . This notation has originally been introduced in the computer science literature and is there in use until today. An alternative, equivalent notion that one finds in most of the biologicalⁱ and bioinformatics literature, is that of a k -mer. Here k refers to the (sub)string length. To be more consistent with the literature, in this chapter we will adopt that notation.

2.1 | Definition and Basic Properties

Remember the following definition, where $s_{k,i} := s[i \dots i+k-1]$ denotes the k -mer starting at index i , $1 \leq i \leq |s|-k+1$:

Definition 2.1 de Bruijn subgraph

Definition 2.2 dimension of a de Bruijn subgraph

Note 2.1 **To 2.1** If the same $(k+1)$ -mer occurs multiple times in s , say m times, we have m parallel edges occurring in the multigraph $DBG(s, k)$.

Kommentar 2.1 Für jeden DB Graph gibt es immer einen Eulerpfad

2.2 | Words with the same $(k+1)$ -mer Profile

Motivated by the fact that the *q*-gram distance does not satisfy the identity property of a metric (see Section 1.1) because there can be distinct sequences with the same *q*-gram profile, we want to determine whether for a given sequence s and integer k there are other sequences that are distinct but have the same $(k+1)$ -mer profile as s . And, if there are, how many?

For $k = 0$, the problem is trivial. Indeed, if a sequence t that is distinct from s corresponds to a permutation of the symbols of s , then s and t clearly share the same 1-mer profile. For $k \geq 1$, an elegant answer can be found with the help of the de Bruijn subgraph $DBG(s, k)$.

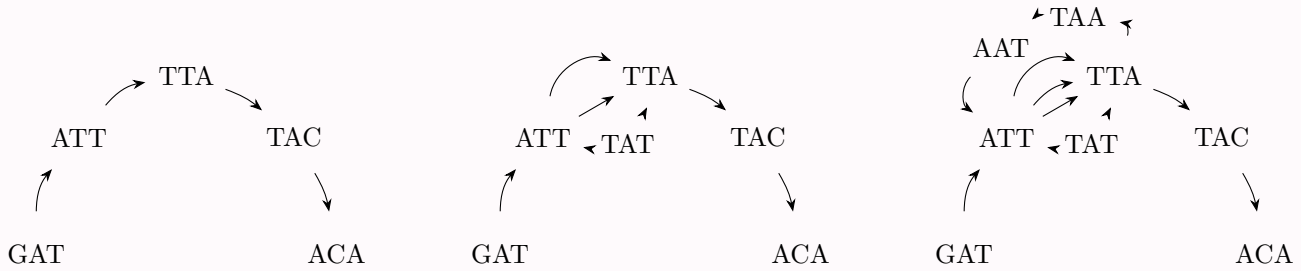
ⁱThe true origin is probably in biochemistry, where notions like monomer or polymer refer to molecular chains of certain lengths.

Clearly, by construction $DBG(s, k)$ forms an Eulerian path p_s , represented by its sequence of vertices

$$s_{k,1}, s_{k,2}, \dots, s_{k,|s|-k+1}.$$

Consequently, $DBG(s, k)$ is connected and balanced and might contain even more than one Eulerian path. In fact, there is a one-to-one correspondence between Eulerian paths in $DBG(s, k)$ and sequences sharing the same $(k+1)$ -mer profile. In other words, a sequence t that is distinct but has the same $(k+1)$ -mer profile as s exists if and only if $DBG(s, k)$ has another Eulerian path p_t corresponding to the sequence of $(k+1)$ -mers of t . Since p_t uses the same edges as p_s in a different order, the path p_t can only exist if $DBG(s, k)$ contains a cycle. Note, however, that the presence of a cycle is not a sufficient condition. Indeed, it is possible that $DBG(s, k)$ contains one or more cycles, but still admits only one Eulerian path. Some examples are given in Figures 2.1 and

Example 2.1



(a) $x = GATTACA, k=3$ - Since this graph contains no cycle, it has only one Eulerian path. There can be no other word sharing the same 4-mer profile.

(b) $x = GATTATTACA, k=3$ - Here the graph contains two cycles, but still only one Eulerian path. There is no other word sharing the same 4-mer profile.

(c) $x = GATTATTAATTACA, k=3$ - This graph contains several cycles and two distinct Eulerian paths. There is exactly one other word with the same 4-mer profile, $GATTAATTATTACA$.

Figure 2.1: Examples of de Bruijn subgraphs for distinct sequences and $k=3$.

Example 2.2 Additional example from lecture

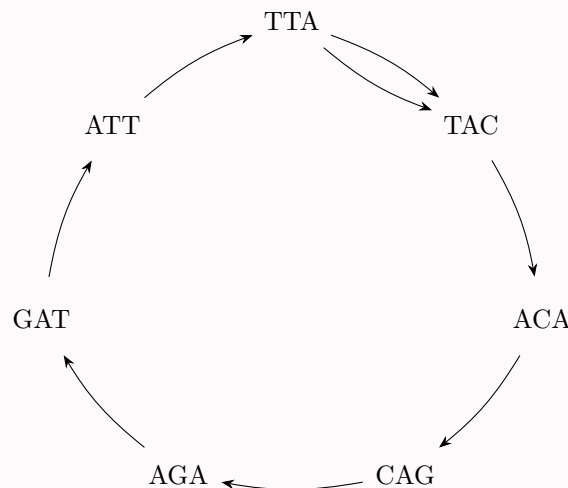


Figure 2.2: It could also be on big circle.

Kommentar 2.2 Reminder that the edges are basically this:



Kommentar 2.3 To figure 2.1 man könnte bei fig 2.1a denken hey bei a kann es nur einen EP geben, weil es kein Kreis ist, und bei 2.1b muss es dann mehrere geben, weil da eine Schleife drin ist, aber NEIN der Pfad ist die Reihenfolge der Knoten und nicht der Kanten, also gibt es nur einen. Man kann nicht unterscheiden, ob man zuerst die obere oder untere Kante der Schleife zuerst gegangen ist. Gibt also nur eine sequenz mit em $k+1$ meer profil. Denn die Sequenz wird ja *aus* den Knoten gemacht, nicht den Kanten ...sozusagen.

We summarize the observations above in the following theorem.

Theorem 2.1 Given a sequence s and an integer $k \geq 1$, the number of distinct sequences that have the same $(k+1)$ -mer profile as s equals the number of distinct Eulerian paths in $DBG(s, k)$. If $DBG(s, k)$ is acyclic, it has a single Eulerian path, and no sequence t exists that is distinct but has the same $(k+1)$ -mer profile as s .

Kommentar 2.4 Auf deutsch: # worte mit gleichen $(k+1)$ -mer Profil wie s = # Euler-Pfade in $DBG(s, k)$

2.3 | Variants of de Bruijn Graphs

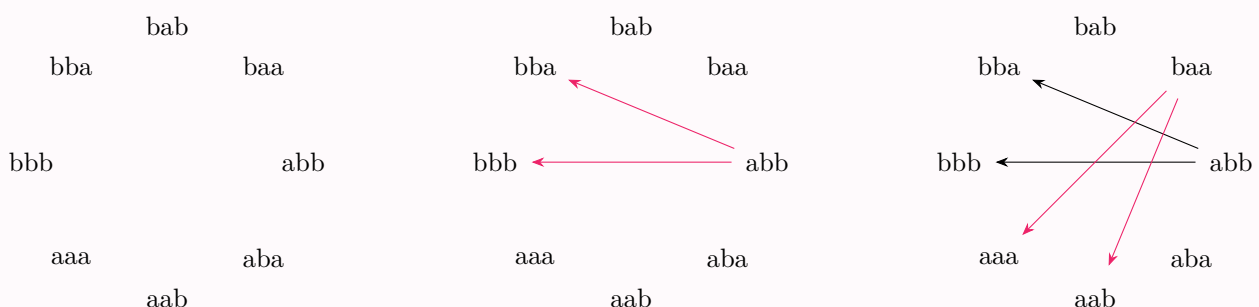
Kommentar 2.5 Der Begriff dbg ist sehr dehnbar, es ist in der Literatur nicht immer das gleiche damit gemeint. Viele sagen dbg, wenn sie eigentlich dbsg meinen.

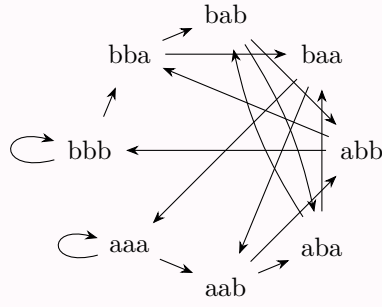
Although, strictly speaking, we consider most of the time **de Bruijn subgraphs** (for a given sequence), often they are just called de Bruijn graphs. In fact, there are more variants of de Bruijn graphs, and often they are also not named very carefully. Here we will discuss them and introduce a precise nomenclature.

Definition 2.3 original de Bruijn graph of dimension k over a finite alphabet Σ of size σ

The graph that contains one vertex for each k -mer (and therefore σ^k vertices) and a directed edge for each overlap of length $k-1$ (and therefore σ^{k+1} edges)

Example 2.3





The **de Bruijn subgraph** for a given sequence s of dimension k is the one that we introduced in Definition 2.1. It has several interesting properties (see Section 2.1) and is used in various contexts in bioinformatics like genome assembly and pangenomics.

Problem 2.1 In most bioinformatics applications where DNA sequence reads come from a sequencing machine, it is not clear from which strand of the DNA double helix a read originates.

$\text{GATT} \rightarrow \text{ATTA} \rightarrow \text{TTAC} \rightarrow \text{TACA}$

$\text{GCTG} \rightarrow \text{CTGT} \rightarrow \text{TGTA}$

Therefore, one prefers to identify a k -mer z with its reverse complement $\overleftarrow{z^{-1}}$ and represent them by the same vertex in the graph:

Example 2.4

$\text{GATT} \rightarrow \text{ATTA} \rightarrow \text{TTAC}$
 $\text{GCTG} \rightarrow \text{CTGT}$

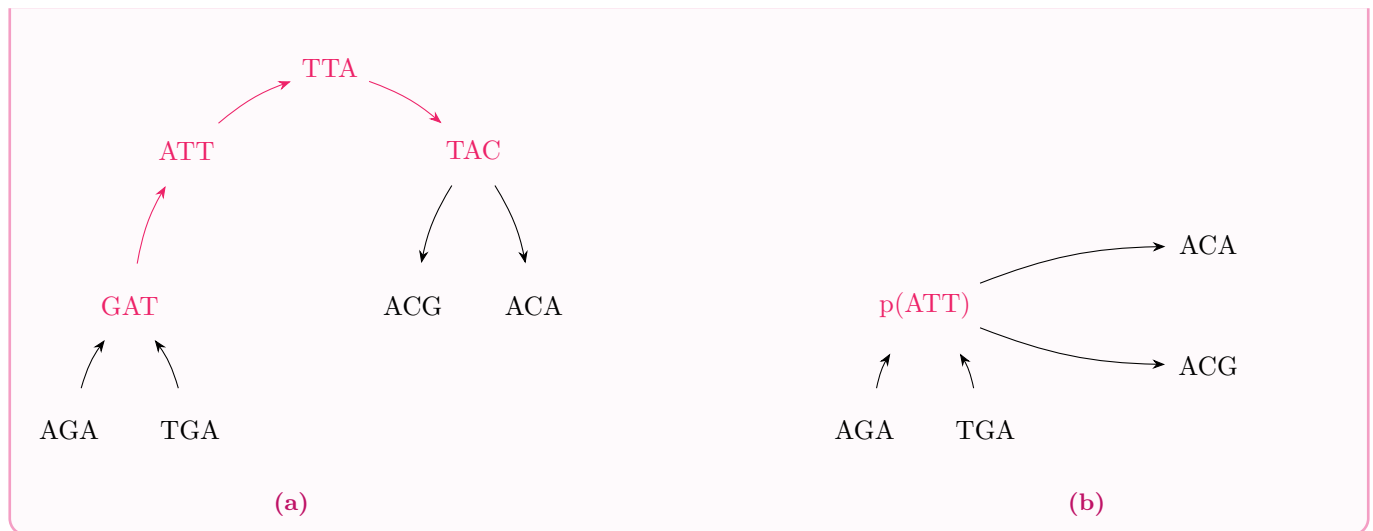
$\rightarrow \text{TACA} / \text{TGTA}$

Definition 2.4 **bidirected de Bruijn subgraph**

Finally, in a (bidirected) **de Bruijn subgraph** one often finds stretches of vertices that have only one successor, the next k -mer in the originating sequence(s). Such a non-branching path can be collapsed into a single vertex (representing a k' -mer for some $k' \geq k$), called unitig, saving space and often computation time.

Definition 2.5 **compacted de Bruijn subgraph**

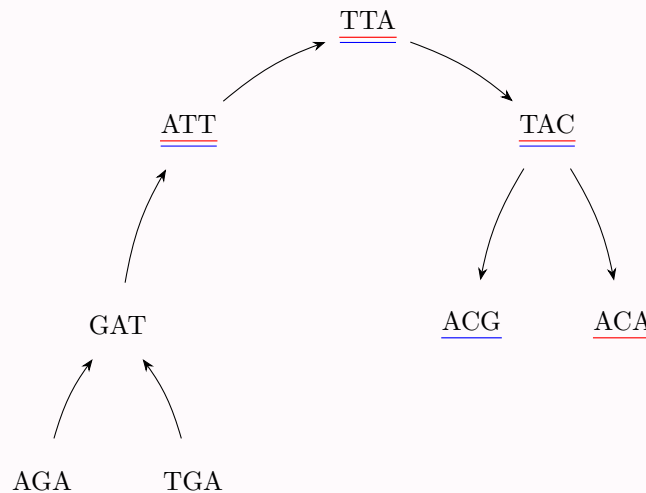
Example 2.5



An additional variant comes into play when the input sequences are partitioned into several groups, for example representing different genomes, and these groups shall also be distinguished in the de Bruijn graph. Then one assigns a color (usually a single bit indicating presence or absence) to each group. A k -mer inherits this color, so that each k -mer gets assigned a color set, indicating in which groups it appears and in which not.

Definition 2.6 colored de Bruijn subgraph

Example 2.6



Problem 2.2 Colored de Bruijn subgraphs form a very powerful data structure, but their memory-efficient implementation is not trivial. In fact, the main challenge is no longer the storage of the k -mers (whose number is limited by σ^k , see above), but the storage of the colors, which may occupy several thousand bits per k -mer in case of large pangenomes.

log der alphabetgröße zur basis 2 ist wie viele bits man braucht zum codieren $k*2$ bits für dbg, aber mit Farben muss man ja noch bits dran hängen, das ist groß und noch ungelöst, denn es ist dann $k*2 + \#$ datensätze

Example 2.7 bsp zu bits: acg codiert als 000110 wenn

A	0	0
C	0	1
G	1	0
T	1	1

2.4 | Implementation of de Bruijn Graphs: Sets of k -mers

Here we consider only the basic variant of **de Bruijn subgraphs**. With a little bit of effort all results can also be generalized to the other graph types.

A simple observation allows to save considerable space: Assume we have a very fast method to test if a certain k -mer is represented in de Bruijn graph $G = \text{DBG}(s, k)$ or not. Then one does not need to store the edges explicitly, for the following reason. When traversing from a vertex representing k -mer au to an adjacent vertex representing k -mer ub ($a, b \in \Sigma, u \in \Sigma^{k-1}$), instead of following the edge $au \rightarrow ub$ explicitly (if it exists), we can instead just test if ub is represented as a vertex in G or not. If the test is successful, we know that the edge exists and therefore can be traversed. If the test is unsuccessful, then the edge does not exist and can not be traversed. Similarly, if we want to find all possible successors of the vertex representing k -mer au , we perform σ tests, one for each possible last character c of the new, overlapping k -mer uc . Especially for small alphabets like DNA, this is not very much work.

Therefore, in (DNA-) bioinformatics applications, a data structure is very popular that represents only a set S of k -mers and offers a very quick presence/absence test, instead of storing a complete graph data structure: the Bloom filter.

The idea is to use one large bit array B of length m , say, as a hash table with all bits initially set to *false* (F), and then have a small number of h different hash functions $f_1, f_2, \dots, f_h$ that map k -mers (respectively, their ranks) to integers in the range $[0, m-1]$. In order to store a k -mer (respectively its rank r), the hash functions are computed and the corresponding bits in B are set:

$$B[f_1(r)] = B[f_2(r)] = \dots = B[f_h(r)] = T.$$

This is repeated for each k -mer.

As always, the hash functions should be independent and uniformly distributed. A very simple one is of the form $f_i(r) = (r^{i+1} \gg k) \bmod m$, where $\gg k$ denotes k -fold bitwise right-shift. There exist, however, much more advanced hash functions in the literature.

In order to test whether a k -mer with rank r has been stored in a Bloom filter B , we perform the same procedure: We evaluate all hash functions in parallel, and if all of them point to T entries in the hash table, then it is very likely that the element r has been stored in B :

$$\text{probably-contains}(B, r) = B[f_1(r)] \wedge B[f_2(r)] \wedge \dots \wedge B[f_h(r)],$$

where $\wedge$ denotes logical AND.

By the combination of several distinct hash functions, a k -mer z (with $\text{rank } r = \text{rank}(z)$) will be reported as a false positive, although it has not been stored in the Bloom filter, only if all hash functions are involved in collisions, i.e., if for each of the h hash values $f_i(r), 1 \leq i \leq h$, some other k -mer $z' \neq z$ produces with some hash function f_i the same hash value ($f_i(\text{rank}(z')) = f_i(r)$) and therefore sets B at this position to T . While this is quite unlikely in practice as long as the hash table is only moderately filled, it can not be completely avoided. Therefore the above function is called “probably-contains” and not “contains”. There are some techniques to handle false positives in certain applications manually, so that they do not distort the result and the Bloom filter becomes exact. Details are omitted here.

Backmatter

Definitionen

q-gram . 3, 5–7, 9, 11

alignment distance The alignment distance of two sequences $x, y \in \Sigma^*$ is defined as $d(x, y) := \min\{\text{cost}(A) : A \in \mathcal{A}^* \text{ is an alignment of } x \text{ and } y\}$. 5

bidirected de Bruijn subgraph . 14

colored de Bruijn subgraph . 15

compact de Bruijn subgraph . 14

de Bruijn graph The graph that contains one vertex for each k-mer (and therefore σ^k vertices) and a directed edge for each overlap of length $k - 1$ (and therefore σ^{k+1} edges). 13

de Bruijn subgraph . iii, v, 11–14, 16

dimension of a de Bruijn subgraph . 11

edit distance test. 5

occurrence count Let $x \in \Sigma^m$ and choose $q \in [1, m]$. The occurrence count of a q-gram $z \in \Sigma^q$ in x is the number

$$\text{Occ}(x, z) := |\{i \in [1, m - q + 1] : x[i \dots i + q - 1] = z\}|$$

. 5

q-gram distance The q-gram distance of $x \in \Sigma^m$ and $y \in \Sigma^n$ is defined for $1 \leq q \leq \min\{m, n\}$ as

$$d_q(x, y) := \sum_{z \in \Sigma^q} |p_q(x)_z - p_q(y)_z|$$

. 5, 6, 8, 9, 11

q-gram profile Let $\sigma = |\Sigma|$. The q-gram profile of x is a σ^q -element vector $p_q(x) := (p_q(x)_z)_{z \in \Sigma^q}$, indexed by all q-grams, defined by $p_q(x)_z := \text{Occ}(x, z)$. 6, 11

ranking algorithm An algorithm that implements a **ranking function**. 6

ranking function A bijective function $r : \mathcal{X} \rightarrow [0, |\mathcal{X}| - 1]$ for a finite set $\mathcal{X}$. iii, 6–10

unranking algorithm An algorithm that implements a **unranking function**. 6

unranking function The inverse of a **ranking function**. iii, 6

List of Figures

2.1 Examples of de Bruijn subgraphs for distinct sequences and $k=3$ 12

2.2 It could also be on big circle. 12